

ACM-ICPC Team Reference Document

Annyal (Anisimov, Basalova, Roitburd)

Contents

1	Templates	1
1.1	Template	1
2	Math	1
2.1	Yura	1
2.2	Convex Hull	2
2.3	Fucntion Eiler	2
2.4	Intersection Segments	3
2.5	Pollard	3
2.6	Primitive _{root}	3
2.7	Diaf	3
2.8	Gauss	3
2.9	Moduls	4
2.10	Closest	4
2.11	Circles	4
2.12	Comb	4
3	Anything	5
3.1	Enumeration _{all_podmasks}	5
3.2	Shpraga _g randy	5
4	Dp	5
4.1	Convex Hull Trick	5
4.2	Sosdp	5
5	Graphs	5
5.1	Bridges And Points	5
5.2	Lca	6
5.3	Kuhn	6
5.4	Min Cost Max Flow	6
5.5	Dinica	7
5.6	Max _n ode _w eight _m atching	7
5.7	Edmonds	7
5.8	2 Sat	8
5.9	Centroid	8
5.10	Hld	8
6	Strings	10
6.1	Pref Func	10
6.2	Z Func	10
6.3	Manaker	10
6.4	Aho Korasik	10
6.5	Palindrom Tree	11
6.6	Suf Array	11
7	Structs	11
7.1	Fenwick _t ree	11
7.2	Sparce _t able	12
7.3	Kurevo	12

1 Templates

1.1 Template

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

#define ordered_set tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>
//order_of_key - сколькоменьше
//find_by_order - элементподномером

auto seed = std::chrono::high_resolution_clock::now().
    time_since_epoch().count();
std::mt19937_64 gen(seed);

int get_random(){
    std::uniform_int_distribution<int> dist(33, 126);
    return dist(gen);
}

std::istream& operator >> (std::istream& in, pii& p)
{
    in >> p.fi >> p.se;
    return in;
}

std::ostream& operator << (std::ostream& os, const pii &p)
{
    return os << p.fi << " " << p.se;
}

void solve() { }

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    int test_cnt = 1;
    cin >> test_cnt;

    while (test_cnt--) {
        solve();
    }
}
```

2 Math

2.1 Yura

$$V = \frac{\pi h^2}{3} (3R - h) \quad // \text{Объем сегмента шара}$$

$$V = \frac{2}{3} \pi R^2 h \quad // \text{Объем сектора шара}$$

$$V_x = \pi \int_a^b [f(x)]^2 dx \quad // \text{Объем тела вращения вокруг OX}$$

$$V_y = 2 \pi \int_a^b x f(x) dx \quad // \text{Объем тела вращения вокруг OY}$$

```
// Инверсия по модулю
int MOD = 998244353;

int power(int a, int b) {
    int res = 1;
    while (b > 0) {
        if (b % 2 == 1) {
            res = (res * a) % MOD;
        }
        a = (a * a) % MOD;
    }
}
```

```

    b /= 2;
}
return res;
}

int inv(int a) {
    return power(a, MOD - 2);
}

// Расширенный алгоритм Евклида
int gcd(int a, int b, int & x, int & y) {
    if (a == 0) {
        x = 0; y = 1;
        return b;
    }
    int x1, y1;
    int d = gcd(b % a, a, x1, y1);
    x = y1 - (b / a) * x1;
    y = x1;
    return d;
}

// Решето
int n;
vector<char> prime(n+1, true); // phi[i] = i;
prime[0] = prime[1] = false;
for (int i=2; i<n; ++i)
    if (prime[i]) // phi[i] == i;
        if (i * 111 * i <= n)
            for (int j=i*i; j<=n; j+=i)
                prime[j] = false; // phi[j] = phi
                [j] * (i - 1) / i

// FFT
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1<<20;

void fft(vector<int> & a, bool invert) {
    int n = (int) a.size();

    for (int i=1, j=0; i<n; ++i) {
        int bit = n >> 1;
        for (; j>=bit; bit>>=1)
            j -= bit;
        j += bit;
        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len=2; len<=n; len<=1) {
        int wlen = invert ? root_1 : root;
        for (int i=len; i<root_pw; i<=1)
            wlen = int(wlen * 111 * wlen % mod);
        for (int i=0; i<n; i+=len) {
            int w = 1;
            for (int j=0; j<len/2; ++j) {
                int u = a[i+j], v = int(a[i+j+
                    len/2] * 111 * w % mod);
                a[i+j] = u+v < mod ? u+v : u-v-
                    mod;
                a[i+j+len/2] = u-v >= 0 ? u-v : u
                    -v+mod;
                w = int(w * 111 * wlen % mod);
            }
        }
    }

    if (invert) {
        int nrev = reverse(n, mod);
        for (int i=0; i<n; ++i)
            a[i] = int(a[i] * 111 * nrev % mod);
    }
}

void multiply(const vector<int> & a, const vector<int> & b
, vector<int> & res) {
    vector<int> fa(a.begin(), a.end()), fb(b.begin(),
        b.end());
    size_t n = 1;
    while (n < max(a.size(), b.size())) n <= 1;
    n <= 1;
    fa.resize(n), fb.resize(n);

    fft(fa, false), fft(fb, false);
    for (size_t i=0; i<n; ++i)
        fa[i] *= fb[i];
    fft(fa, true);

```

```

        res.resize(n);
        for (size_t i=0; i<n; ++i)
            res[i] = int(fa[i].real() + 0.5);
    }

```

2.2 Convex Hull

```

vector<pair<double,double>> vyp(vector<pair<double,double>>&
a){
    int ind = 0;
    int n = (int)a.size();
    for(int i=0; i<n; i++){
        if(a[i].fi < a[ind].fi){
            ind = i;
            continue;
        }
        if(a[i].fi == a[ind].fi && a[i].se < a[ind].se){
            ind = i;
            continue;
        }
    }
    p0 = a[ind];
    swap(a[0], a[ind]);
    sort(a.begin() + 1, a.end(), comp);
    vector<pair<double,double>>res;
    res.pb(a[0]);
    for(int i=1; i<n; i++){
        pair<double,double>r1,r2;
        while((int)res.size() > 2){
            r1 = {a[i].fi - res.back().fi, a[i].se - res.back
                ().second};
            r2 = {res.back().fi - res[(int)res.size()-2].fi,
                res.back().se - res[(int)res.size()-2].se};
            if(r1.fi*r2.se - r1.se*r2.fi >= 0){
                res.pop_back();
            }
            else break;
        }
        res.pb(a[i]);
    }
    return res;
}

```

2.3 Fucntion Euler

```

// for one
int func(int n){
    int ans = n;
    for(int i=2; i<=n; i++){
        if(n % i == 0){
            while(n%i == 0){
                n/=i;
            }
            ans -= ans/i;
        }
    }
    if(n > 1)ans -=ans/(n);
    return ans;
}

//for many
vi p;
vi d;

void er(int n){
    d.assign(n+1,0);
    for(int i=2; i<=n; i++){
        if(!d[i]){
            p.pb(i);
            d[i] = i;
            for(int j=i*i; j<=n; j+=i)d[j] = i;
        }
    }
}

vi f;

int func(int n){
    int nn = n;
    int x = d[n];

```

```

int ans = x-1;
n/=x;
while(n%x == 0){
    n/=x;
    ans *=x;
}
ans *= f[n];
f[nn] = ans;
return ans;
}

```

2.4 Intersection Segments

```

struct pt {
    int x, y;
};

inline int area (pt a, pt b, pt c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

inline bool intersect_1 (int a, int b, int c, int d) {
    if (a > b) swap (a, b);
    if (c > d) swap (c, d);
    return max(a,c) <= min(b,d);
}

bool intersect (pt a, pt b, pt c, pt d) {
    return intersect_1 (a.x, b.x, c.x, d.x)
        && intersect_1 (a.y, b.y, c.y, d.y)
        && area(a,b,c) * area(a,b,d) <= 0
        && area(c,d,a) * area(c,d,b) <= 0;
}

```

2.5 Pollard

```

typedef long long ll;

inline ll f(ll x, ll n) { return (__int128_t) (x + 1) * (x + 1) % n; }

ll find_divisor(ll n, ll seed = 1) {
    ll x = seed, y = seed;
    ll d = 1;
    while (d == 1 || d == n) {
        // двигаем первый указатель на шаг
        y = f(y);
        // авторой -- на два
        x = f(f(x));
        // пытаемся найти общий делитель
        d = gcd(abs(x - y), n);
    }
    return d;
}

```

2.6 Primitive root

```

//Первообразный корень по модулю p существует тогда и только тогда,
//когда p является либо степенью нечётного простого,
//либо двойной степенью простого,
//а также в случаях n=1, n=2, n=4.

int powmod (int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int (res * 1ll * a % p), --b;
        else
            a = int (a * 1ll * a % p), b >>= 1;
    return res;
}

int generator (int p) {
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {

```

```

            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=p; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= powmod (res, phi / fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}

```

2.7 Diaf

```

int gcd (int a, int b, int &x, int &y) {
    if (a == 0) {
        x = 0;
        y = 1;
        return b;
    }
    int x1, y1;
    int d = gcd (b % a, a, x1, y1);
    x = y1 - (b / a) * x1;
    y = x1;
    return d;
}

bool find_any_solution (int a, int b, int c, int &x0, int &y0, int &g) {
    g = gcd (abs (a), abs (b), x0, y0);
    if (c % g != 0)
        return false;
    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 *= -1;
    if (b < 0) y0 *= -1;
    return true;
}

// x = x0 + k * (b / g)
// y = y0 - k * (a / g)

```

2.8 Gauss

```

int g[3003][3003];
int vv[3003];
int ans[3003];
int ogr[3003];

int gauss(int n)
{
    for(int i = 0; i < n; i++)
    {
        int mx = g[vv[i]][i], pos = i;
        for(int j = i; j < n; j++)
            if(g[vv[j]][i] > mx)
            {
                mx = g[vv[j]][i];
                pos = j;
            }
        swap(vv[i], vv[pos]);
        assert(mx);
        long long tmp = binpow(mx);
        int T = 0;
        for(int j = i + 1; j < n; j++)
            if(g[vv[j]][i]) ogr[T++] = j;
        for(int j = n; j >= i; j--)
        {
            g[vv[i]][j] = g[vv[i]][j] * tmp % mod;
            if(g[vv[i]][j])
            {
                for(int u = 0; u < T; u++) {
                    int t = ogr[u];
                    long long value = g[vv[t]][j] - 1ll * g[vv[i]][j] * g[vv[t]][i] % mod + mod;
                    if (value >= mod) {
                        value -= mod;
                    }
                    g[vv[t]][j] = value;
                }
            }

```

```

    }
    }
}
ans[n - 1] = g[vv[n - 1]][n];
for(int i = n - 2; i >= 0; i--)
{
    int tmp = g[vv[i]][n];
    for(int j = i + 1; j < n; j++)
        tmp = (011 + mod + tmp - 111 * g[vv[i]][j] * ans[
            j] % mod) % mod;
    ans[i] = tmp;
}
return ans[0];
}

```

2.9 Moduls

```

r[1] = 1;
for (int i=2; i<m; ++i)
    r[i] = (m - (m/i) * r[m%i] % m) % m;

```

2.10 Closest

```

int n, a[MAXN];
Point p[MAXN], tmp[MAXN], scan[MAXN];
ll ans = INF;
int sz;
void relax (Point &a, Point &b) {
    ll res = (a - b).sqrLen ();
    if (res < ans) ans = res;
}
bool cmpy (Point a, Point b) {
    return a.y < b.y;
}
void f (int l, int r) { // Half - interval [l, r)
    if (r - l < 5) {
        for (int i = l; i < r; i++)
            for (int j = i + 1; j < r; j++)
                relax (p[i], p[j]);
        return sort (p + l, p + r, cmpy);
    }
    int m = (l + r) >> 1;
    int x = p[m].x;
    f(l, m); f(m, r);
    merge (p + l, p + m, p + m, p + r, tmp, cmpy);
    sz = 0;
    for (int i = l; i < r; i++) {
        p[i] = tmp[i - l];
        if (sqr (x - p[i].x) < ans) {
            for (int j = sz - 1; j > -1 && sqr (p[i]
                ].y - (ll) scan [j].y) < ans ; j
                --)
                relax (p[i], scan [j]);
            scan [sz ++] = p[i];
        }
    }
}

```

2.11 Circles

```

void out(long double x, long double y){
    cout << x << ' ' << y << '\n';
}

void Solve(){
    int x1, y1, r1;
    int x2, y2, r2;

    cin >> x1 >> y1 >> r1;
    cin >> x2 >> y2 >> r2;

    int D = sqr(x1 - x2) + sqr(y1 - y2);

    if(D == 0 && r1 == r2)
        cout << "-1\n";
    else if (D > sqr(r1 + r2) || D < sqr(r2 - r1))
        cout << "0\n";
    else{

```

```

        int a = 2 * (x2 - x1);
        int b = 2 * (y2 - y1);
        int c = sqr(r2) - sqr(r1) - sqr(x2 - x1) - sqr(y2 -
            y1);

        int T = 4 * D;
        long double d = sqrt1(D) * 2;
        long long D2 = 411 * D * r1 * r1 - (11)c * c;
        long double d2 = sqrt1(D2) / (4 * D);
        long double hx = (long double)(111 * T * x1 - (11)c
            * a) / T;
        long double hy = (long double)(111 * T * y1 - (11)c
            * b) / T;

        if(D2 == 0){
            cout << "1\n";
            out(hx, hy);
        }else{
            cout << "2\n";
            pair<long double, long double> ans1 = {hx - b *
                d2, hy + a * d2};
            pair<long double, long double> ans2 = {hx + b *
                d2, hy - a * d2};

            if (ans1.first > ans2.first || ans1.first == ans2
                .first && ans1.second > ans2.second) {
                swap(ans1, ans2);
            }

            out(ans1.first, ans1.second);
            out(ans2.first, ans2.second);
        }
    }
}

```

2.12 Comb

```

int gray_code (int n) {
    return n ^ (n >> 1);
}

int count_bits (int n) {
    int res = 0;
    for (; n; n >>= 1)
        res += n & 1;
    return res;
}

void all_combinations (int n, int k) {
    for (int i = 0; i < (1 << n); i++) {
        int cur = gray_code (i);
        if (count_bits(cur) == k) {
            for (int j = 0; j < n; j++) {
                if (cur & (1 << j))
                    cout << j + 1;
            }
            cout << "\n";
        }
    }
}

vector<int> ans;

void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0)
        return;

    if (!n) {
        for (int i = 0; i < idx; ++i) {
            if (ans[i])
                cout << i + 1;
        }
        cout << "\n";
        return;
    }

    ans[idx] = rev;
    gen(n - 1, k - rev, idx + 1, false);
    ans[idx] = !rev;
    gen(n - 1, k - !rev, idx + 1, true);
}

void all_combinations(int n, int k) {
    ans.resize(n);
    gen(n, k, 0, false);
}

```

3 Anything

3.1 Enumeration $all_p od masks$

```
//0(3^n)
for (int m=0; m<(1<<n); ++m)
    for (int s=m; s; s=(s-1)&m)
        ... использования им ...
```

3.2 Shpraga $randy$

```
//Решение нима
//Теорема.
    Текущий игрок имеет выигрышную стратегию тогда и только тогда,
    когда XOR сумма – размеров кучек отлична от нуля.

//Теорема Шпрага-Гранди-.
//Рассмотрим любое состояние в
    некоторой равноправной игре двух игроков.
//Пусть из него есть переходы в некоторые состояния v_i.
//Утверждается, что состоянию v
    этой игры можно поставить в соответствие кучку некоторого размера
    x.
//Это число называется значением Шпрага-Гранди – состояния v.
//Более того, это число
    можно находить следующим рекурсивным образом:
//посчитаем значение Шпрага-Гранди – x_i по каждому переходу (v,
    v_i), и тогда выполняется:
// x = mex{x1, x2, ..., xk}.
//Итак, чтобы посчитать функцию Шпрага-Гранди –
    для текущего состояния некоторой игры, нужно:
//Выписать все возможные переходы из текущего состояния.
//Каждый такой переход может вести либо в одну игру,
    либо в сумму независимых игр.
//В первом случае –
    просто посчитаем функцию Гранди рекурсивно для того нового состояния
    .
//Во втором случае,
    когда переход из текущего состояния приводит к сумме нескольких независимых игр –
    рекурсивно посчитаем для каждой из этих игр функцию Гранди,
    а затем скажем, что функция Гранди суммы игр
//равна XOR сумме – значений этих игр.
//После того,
    как мы посчитали функцию Гранди для каждого возможного перехода –
    считаем mex от этих значений, и найденное число –
    есть искомое значение Гранди для текущего состояния.
```

4 Dp

4.1 Convex Hull Trick

```
struct line {
    int k, b;
    line() {}
    line(int a, int _b) { k = a, b = _b; }
    int get(int x) { return k * x + b; }
};

vector<line> lines; // храним прямые нижней огибающей
vector<int> dots; // храним координаты –
    точек нижней огибающей
// ^ первое правило шестенных чисел
// считаем, что в dots лежат координаты нижней огибающей

int cross(line a, line b) { // считаем точку пересечения
    // считаем a.k > b.k
    int x = (b.b - a.b) / (a.k - b.k);
    if (b.b < a.b) x--; //
        бремся со округлением отрицательных чисел
    return x;
}

void add(line cur) {
```

```
while (lines.size() && lines.back().get(dots.back()) >
    cur.get(dots.back())) {
    lines.pop_back();
    dots.pop_back();
}
if (lines.empty())
    dots.push_back(-inf);
else
    dots.push_back(cross(lines.back(), cur));
lines.push_back(cur);
}

int get(int x) {
    int pos = lower_bound(dots.begin(), dots.end(), x) -
        dots.begin() - 1;
    return lines[pos].get(x);
}
```

4.2 Sospd

```
/*
problem:
Given a fixed array A of 2N integers, we need to calculate
x function F(x) = Sum of all A[i] such that x & i = i,
i.e., i is a subset of x.
*/

//iterative version
for (int mask = 0; mask < (1<<N); ++mask) {
    dp[mask][1] = A[mask]; //handle base case
    separately (leaf states)
    for (int i = 0; i < N; ++i) {
        if (mask & (1<<i))
            dp[mask][i] = dp[mask][i-1] + dp[mask
                ^ (1<<i)][i-1];
        else
            dp[mask][i] = dp[mask][i-1];
    }
    F[mask] = dp[mask][N-1];
}
//memory optimized, super easy to code.
for (int i = 0; i < (1<<N); ++i)
    F[i] = A[i];
for (int i = 0; i < N; ++i) for (int mask = 0; mask < (1<<N);
    ++mask) {
    if (mask & (1<<i))
        F[mask] += F[mask ^ (1<<i)];
    }
}
```

5 Graphs

5.1 Bridges And Points

```
//bridges
const int maxn = 1e5;

bool used[maxn];
int h[maxn], d[maxn];

void dfs(int v, int p = -1) {
    used[v] = true;
    d[v] = h[v] = (p == -1 ? 0 : h[p] + 1);
    for (int u : g[v]) {
        if (u != p) {
            if (used[u]) // если ребро обратное
                d[v] = min(d[v], h[u]);
            else { // если ребро прямое
                dfs(u, v);
                d[v] = min(d[v], d[u]);
                if (h[v] < d[u]) {
                    // если не ядро другим путем добраться в
                    // или выше,
                    // то ребро (v, u) – мост
                }
            }
        }
    }
}

//points_of_articulations
```

```

void dfs(int v, int p = -1) {
    used[v] = 1;
    d[v] = h[v] = (p == -1 ? 0 : h[p] + 1);
    int children = 0;
    for (int u : g[v]) {
        if (u != p) {
            if (used[u])
                d[v] = min(d[v], h[u]);
            else {
                dfs(u, v);
                d[v] = min(d[v], d[u]);
                if (h[v] <= d[u] && p != -1) { // корень(p ==
                    -1) обрабатываем отдельно
                    // v -- точкасочленения
                    // это(
                        условиеможетвыполнитьсямногогоразддляразныхдетей
                    )
                }
                children++;
            }
        }
    }
    if (p == -1 && children > 1) {
        // v -- кореньиточкасочленения
    }
}

```

5.2 Lca

```

//prep
int up[maxn][logn];

void dfs(int v) {
    for (int l = 1; l < logn; l++)
        up[v][l] = up[up[v][l-1]][l-1];
    tin[v] = t++;
    for (int u : g[v]) {
        up[u][0] = v;
        dfs(u);
    }
    tout[v] = t;
}

//get
int lca(int v, int u) {
    if (a(v, u)) return v;
    if (a(u, v)) return u;
    for (int l = logn-1; l >= 0; l--)
        if (!ancestor(up[v][l], u))
            v = up[v][l];
    return up[v][0];
}

//O(1)

vector<vector<int>> gr;
vector<int> a, fst, g;

void dfs(ll v, ll h){
    fst[v] = a.size();
    a.pb(h);
    g.pb(v);
    for(auto to : gr[v]){
        dfs(to, h+1);
        a.pb(h);
        g.pb(v);
    }
}

solve(){
    dfs(0, 0);
    sparse table(a);
    if(fst[u] > fst[v]) swap(u, v);
    ll lca = g[table.get_min(fst[u], fst[v])];
}

```

5.3 Kuhn

```

//prep
const int maxn;
vector<int> g[maxn]; //
    будемхранитьтолькокорёбраизлевойдоливправую
int mt[maxn]; // скакойвершинойсматченавершинаправойдоли
    (-1, еслинискокой)
bool used[maxn]; // вспомогательныймассивдляпоискапутидfs
    om-

// dfs возвращает, можнолинайтипутьизвершиныv
// вкакуюнибуть- вершинуправойдоли
// еслиможно, тоещёипроводитчередование
bool dfs(int v) {
    if (used[v])
        return false;
    used[v] = true;
    for (int u : g[v]) {
        // есливершинасвободна, томожносразуейсоединиться
        // еслионазанята, тосейможносоединитьсятолькотогда,
        // когдаизеётекущейпарыможнонайтикакуюнибуть-
        // другуювершину
        if (mt[u] == -1 || dfs(mt[u])) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}

//doing
memset(mt, -1, sizeof mt);
for (int i = 0; i < n; i++) {
    memset(used, 0, sizeof mt);
    if (dfs(i))
        cnt++;
}

```

5.4 Min Cost Max Flow

```

const int maxn = 305, inf = 1e9;

int n;
int cost[maxn][maxn], cap[maxn][maxn];
int d[maxn], pot[maxn], par[maxn];

bool dijkstra(int s, int t) {
    used[maxn] = {0};

    fill(d, d+n, inf);
    d[s] = 0;

    while (1) {
        int v = -1;
        for (int u = 0; u < n; u++)
            if (!used[u] && (v == -1 && d[u] < d[v]))
                v = u;
        if (v == -1 || d[v] == inf)
            break;
        used[v] = 1;
        for (int u = 0; u < n; u++) {
            int w = cost[v][u] + pot[v] - pot[u];
            if (cap[v][u] && d[u] > d[v] + w) {
                d[u] = d[v] + w;
                par[u] = v;
            }
        }
    }

    return d[t] < inf;
}

int mincost_maxflow(int s, int t) {
    int ans = 0;
    while (dijkstra(s, t)) {
        memcpy(pot, d, sizeof(d));
        int delta = inf;
        for (int v = t; v != s; v = par[v])
            delta = min(delta, cap[par[v]][v]);
        for (int v = t; v != s; v = par[v]) {
            cap[par[v]][v] -= delta;
            cap[v][par[v]] += delta;
            ans += cost[par[v]][v]*delta;
        }
    }
    return ans;
}

```

5.5 Dinica

```
const int MAXN = ...; // число вершин
const int INF = 1000000000; // константа бесконечность-

struct edge {
    int a, b, cap, flow;
};

int n, s, t, d[MAXN], ptr[MAXN], q[MAXN];
vector<edge> e;
vector<int> g[MAXN];

void add_edge (int a, int b, int cap) {
    edge e1 = { a, b, cap, 0 };
    edge e2 = { b, a, 0, 0 };
    g[a].push_back ((int) e.size());
    e.push_back (e1);
    g[b].push_back ((int) e.size());
    e.push_back (e2);
}

bool bfs() {
    int qh=0, qt=0;
    q[qt++] = s;
    memset (d, -1, n * sizeof d[0]);
    d[s] = 0;
    while (qh < qt && d[t] == -1) {
        int v = q[qh++];
        for (size_t i=0; i<g[v].size(); ++i) {
            int id = g[v][i],
                to = e[id].b;
            if (d[to] == -1 && e[id].flow < e[id].cap) {
                q[qt++] = to;
                d[to] = d[v] + 1;
            }
        }
    }
    return d[t] != -1;
}

int dfs (int v, int flow) {
    if (!flow) return 0;
    if (v == t) return flow;
    for (; ptr[v]<(int)g[v].size(); ++ptr[v]) {
        int id = g[v][ptr[v]],
            to = e[id].b;
        if (d[to] != d[v] + 1) continue;
        int pushed = dfs (to, min (flow, e[id].cap - e[id].flow));
        if (pushed) {
            e[id].flow += pushed;
            e[id^1].flow -= pushed;
            return pushed;
        }
    }
    return 0;
}

int dinic() {
    int flow = 0;
    for (;;) {
        if (!bfs()) break;
        memset (ptr, 0, n * sizeof ptr[0]);
        while (int pushed = dfs (s, INF))
            flow += pushed;
    }
    return flow;
}
```

5.6 $\text{Max}_{node_weighted_matching}$

// Дан двудольный граф G.
 Для каждой вершины первой доли указан её вес.
 // Требуется найти паросочетание наибольшего веса, т.е.
 с наибольшей суммой весов насыщенных вершин.

```
int n;
vector<vector<int>> g (n);
vector<int> used (n);
vector<int> order (n); // список вершин,
    отсортированный по весу
... чтение...
```

```
for (int i=0; i<n; ++i) {
    int v = order[i];
    used.assign (n, false);
    try_kuhn (v);
}
```

5.7 Edmonds

// Дан неориентированный невзвешенный граф G с n вершинами.
 Требуется найти в нём наибольшее паросочетание,
 // т.е. такое наибольшее (по мощности) множество его рёбер,
 чьи концы никакие два не образуют выбранных их инцидентных друг другу.

// Общая схема

```
void edmonds() {
    for (int i=0; i<n; ++i)
        if (вершина( i не в паросочетании) {
            int last_v = find_augment_path (i);
            if (last_v != -1)
                выполнить чередование вдоль пути из i в last_v;
        }
}
```

int find_augment_path (int root) { обход в ширину

```
    int v = текущая вершина; перебрать все рёбра из
        v, если обнаружил цикл нечётной длины
        /
        / сжать его, если пришл в свободную вершину
        / return
        / если пришл в не свободную вершину
        /
        / то добавить в очередь смежную ей в паросочетании

    return -1;
}
```

// сжатие цветка

```
int v, u; // ребро (v,u),
    при рассмотрении которого было обнаружено цветковое
int b = lca (v, u);
memset (blossom, 0, sizeof blossom);
mark_path (v, b, u);
mark_path (u, b, v);

void mark_path (int v, int b, int children) {
    while (base[v] != b) {
        blossom[base[v]] = blossom[base[match[v]]] =
            true;
        p[v] = children;
        children = match[v];
        v = p[match[v]];
    }
}
```

int find_path (int root) {

```
    memset (used, 0, sizeof used);
    memset (p, -1, sizeof p);
    for (int i=0; i<n; ++i)
        base[i] = i;

    used[root] = true;
    int qh=0, qt=0;
    q[qt++] = root;
    while (qh < qt) {
        int v = q[qh++];
        for (size_t i=0; i<g[v].size(); ++i) {
            int to = g[v][i];
            if (base[v] == base[to] || match[v] == to) continue;
            if (to == root || match[to] != -1 && p[match[to]] != -1) {
                int curbase = lca (v, to);
                memset (blossom, 0, sizeof blossom);
                mark_path (v, curbase, to);
                mark_path (to, curbase, v);
                for (int i=0; i<n; ++i)
                    if (blossom[base[i]]) {
                        base[i] = curbase;
                        if (!used[i]) {
                            used[i] = true;

```

```

        }
        q[qt++] = i;
    }
}
else if (p[to] == -1) {
    p[to] = v;
    if (match[to] == -1)
        return to;
    to = match[to];
    used[to] = true;
    q[qt++] = to;
}
}
return -1;
}

const int MAXN = ...; //
максимально возможное число вершин в входном графе

int n;
vector<int> g[MAXN];
int match[MAXN], p[MAXN], base[MAXN], q[MAXN];
bool used[MAXN], blossom[MAXN];

...

int main() {
    ... чтение графа ...

    memset (match, -1, sizeof match);
    for (int i=0; i<n; ++i)
        if (match[i] == -1) {
            int v = find_path (i);
            while (v != -1) {
                int pv = p[v], ppv = match[pv];
                match[v] = pv, match[pv] = v;
                v = ppv;
            }
        }
}

```

5.8 2 Sat

```

int n;
vector<vector<int>> g, gt;
vector<bool> used;
vector<int> order, comp;

void dfs1 (int v) {
    used[v] = true;
    for (size_t i=0; i<g[v].size(); ++i) {
        int to = g[v][i];
        if (!used[to])
            dfs1 (to);
    }
    order.push_back (v);
}

void dfs2 (int v, int cl) {
    comp[v] = cl;
    for (size_t i=0; i<gt[v].size(); ++i) {
        int to = gt[v][i];
        if (comp[to] == -1)
            dfs2 (to, cl);
    }
}

int main() {
    ... чтение, графа g, построение графа gt ...

    used.assign (n, false);
    for (int i=0; i<n; ++i)
        if (!used[i])
            dfs1 (i);

    comp.assign (n, -1);
    for (int i=0, j=0; i<n; ++i) {
        int v = order[n-i-1];
        if (comp[v] == -1)
            dfs2 (v, j++);
    }

    for (int i=0; i<n; ++i)
        if (comp[i] == comp[i^1]) {

```

```

        puts ("NO SOLUTION");
        return 0;
    }
    for (int i=0; i<n; ++i) {
        int ans = comp[i] > comp[i^1] ? i : i^1;
        printf ("%d ", ans);
    }
}

```

5.9 Centroid

```

int l = 179; // необходимая длина
int ans = 0;

// наменьяяно удалять вершины: заведем массив used --
// была ли вершина удалена
bool used[maxn];
int s[maxn]; // размеры поддеревьев

void sizes (int v, int p) {
    s[v] = 1;
    for (int u : g[v])
        if (u != p && !used[u])
            sizes(u, v), s[v] += s[u];
}

int centroid (int v, int p, int n) {
    for (int u : g[v])
        if (u != p && !used[u] && s[u] > n/2)
            return centroid(u, v, n);
    return v;
}

// записывает в t[] глубины вершин
void dfs (int v, int p, int d, vector<int> &t) {
    t.push_back(d);
    for (int u : g[v])
        if (u != p && !used[u])
            dfs(u, v, d + 1, t);
}

void solve (int v) {
    /* единственный < зависящий от конкретной задачи код */
    sizes(v);
    vector<int> d(s[v], 0);
    d[0] = 1;
    for (int u : g[v]) {
        if (!used[u]) {
            vector<int> t;
            dfs(u, v, 1, t);
            for (int x : t)
                if (x <= 1)
                    ans += d[1-x];
            for (int x : t)
                d[x]++;
        }
    }
    /* единственный </ зависящий от конкретной задачи код */

    used[v] = 1;
    for (int u : g[v])
        if (!used[u])
            solve(centroid(u, v, s[u]));
}

```

5.10 Hld

```

#include<bits/stdc++.h>
#define f first
#define s second
#define ll long long
#define int short
using namespace std;

vector<int> g[10004];
int tin[10004];
int tout[10004];
int T = 1, C = 1;
int m[10004];
int p[10004][22];
int w[10004];

```

```

int link[10004];
int comp[10004];
int dist[10004];
vector<int> path[10004];
vector<pair<int, int>> tree[10004];
vector<int> push[10004];
int k;
int was[10004];

void build(int ind, vector<int>& v)
{
    //if(tree[ind].empty())
    //{
        tree[ind].clear();
        push[ind].clear();
        m[ind] = 1;
        while(m[ind] < (int)v.size()) m[ind] *= 2;
        tree[ind].resize(m[ind] * 2, {0, 0});
        push[ind].resize(m[ind] * 4, 0);
    //}
    //else
    //{
        // fill(tree[ind].begin(), tree[ind].end(), make_pair
        // (0, 0));
        // fill(push[ind].begin(), push[ind].end(), 0);
    // }
}

void do_push(int ind, int v, int l, int r)
{
    push[ind][v * 2] += push[ind][v];
    push[ind][v * 2 + 1] += push[ind][v];
    if(tree[ind][v].s == 0) tree[ind][v].s = (r - l + 1);
    tree[ind][v].f += push[ind][v];
    push[ind][v] = 0;
}

void add(int ind, int l, int r, int ql, int qr, int v, int val)
{
    do_push(ind, v, ql, qr);
    if(ql > r || qr < l) return;
    if(ql >= l && qr <= r)
    {
        push[ind][v] += val;
        do_push(ind, v, ql, qr);
        return;
    }
    int qm = (ql + qr) / 2;
    add(ind, l, r, ql, qm, v * 2, val);
    add(ind, l, r, qm + 1, qr, v * 2 + 1, val);
    if(tree[ind][v * 2].f > tree[ind][v * 2 + 1].f)
        tree[ind][v] = {tree[ind][v * 2].f, tree[ind][v * 2].s};
    else if(tree[ind][v * 2].f < tree[ind][v * 2 + 1].f)
        tree[ind][v] = {tree[ind][v * 2 + 1].f, tree[ind][v * 2 + 1].s};
    else tree[ind][v] = {tree[ind][v * 2 + 1].f, tree[ind][v * 2 + 1].s + tree[ind][v * 2].s};
}

int get(int ind, int l, int r, int ql, int qr, int v)
{
    do_push(ind, v, ql, qr);
    if(ql > r || qr < l) return 0;
    if(tree[ind][v].f < k) return 0;
    if(ql >= l && qr <= r)
    {
        if(tree[ind][v].f == k)
            return tree[ind][v].s;
        else return 0;
    }
    int qm = (ql + qr) / 2;
    return get(ind, l, r, ql, qm, v * 2)
        + get(ind, l, r, qm + 1, qr, v * 2 + 1);
}

int upper(int a, int b)
{
    if(tin[b] <= tin[a] && tout[b] >= tout[a]) return 1;
    return 0;
}

int lca(int a, int b)
{
    if(upper(a, b)) return b;
    if(upper(b, a)) return a;
    for(int st = 15; st >= 0; st--)
        if(p[b][st] > 0 && !upper(a, p[b][st]))

```

```

        b = p[b][st];
        return p[b][0];
    }

int dfs(int v, int pp = -1)
{
    p[v][0] = pp;
    w[v] = 0;
    for(int i = 1; i < 16; i++)
    {
        if(p[v][i - 1] == -1)
            p[v][i] = -1;
        else
            p[v][i] = p[p[v][i - 1]][i - 1];
    }
    tin[v] = T++;
    for(auto to: g[v])
    {
        if(to == pp) continue;
        w[v] += dfs(to, v);
    }
    tout[v] = T++;
    return w[v] + 1;
}

void hld(int v, int pp = -1, int d = 0)
{
    int mx = -1, poz = -1;
    for(auto to: g[v])
    {
        if(to == pp) continue;
        if(w[to] > mx)
        {
            mx = w[to];
            poz = to;
        }
    }
    for(auto to: g[v])
    {
        if(to == pp) continue;
        if(poz == to)
        {
            path[comp[v]].push_back(to);
            dist[to] = d + 1;
            comp[to] = comp[v];
            hld(to, v, d + 1);
        }
        else
        {
            C++;
            comp[to] = C;
            link[C] = v;
            path[C].push_back(to);
            dist[to] = 0;
            hld(to, v, 0);
        }
    }
}

void dfs_add(int v, int val, int cur)
{
    int cnt = 0;
    while(v >= 0)
    {
        int c = comp[v];
        if(was[c] < cur)
        {
            build(c, path[c]);
            was[c] = cur;
        }
        add(c, 0, dist[v], 0, m[c] - 1, 1, val);
        v = link[c];
    }
}

int dfs_get(int v)
{
    int cnt = 0;
    while(v >= 0)
    {
        int c = comp[v];
        cnt += get(c, 0, dist[v], 0, m[c] - 1, 1);
        v = link[c];
    }
    return cnt;
}

void solve()

```

```

{
    int n;
    cin >> n;
    for(int i = 0; i < n - 1; i++)
    {
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    path[C].push_back(0);
    link[C] = -1;
    comp[0] = C;
    dist[0] = 0;
    dfs(0);
    hld(0);
    int q;
    cin >> q;

    for(int i = 0; i < q; i++)
    {
        cin >> k;
        int cnt = 0;
        int a, b, lc;
        for(int j = 0; j < k; j++)
        {
            cin >> a >> b;
            a--;
            b--;
            dfs_add(a, 1, i + 1);
            dfs_add(b, 1, i + 1);
            lc = lca(a, b);
            dfs_add(lc, -1, i + 1);
            if(p[lc][0] >= 0)
                dfs_add(p[lc][0], -1, i + 1);
        }
        cnt += dfs_get(a);
        cnt += dfs_get(b);
        cnt -= dfs_get(lc);
        cout << cnt << "\n";
    }
    for(int i = 0; i < n; i++)
    {
        g[i].clear();
        path[i].clear();
        w[i] = 0;
    }
    for(int i = 1; i <= C; i++)
    {
        tree[i].clear();
        push[i].clear();
        was[i] = 0;
    }
    T = 0;
    C = 1;
}

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int T;
    cin >> T;
    int t = 1;
    while(t <= T)
    {
        cout << "Case " << t << ": " << "\n";
        solve();
        t++;
    }
}

```

6 Strings

6.1 Pref Func

```

vector<int> prefix_function(string s) {
    int n = (int) s.size();
    vector<int> p(n, 0);
    for (int i = 1; i < n; i++) {
        // префикс-функция точно не больше этого значения + 1

```

```

        int cur = p[i - 1];
        // уменьшаем cur значение, пока новый символ не совпадет
        while (s[i] != s[cur] && cur > 0)
            cur = p[cur - 1];
        // здесь либо s[i] == s[cur], либо cur == 0
        if (s[i] == s[cur])
            p[i] = cur + 1;
    }
    return p;
}

```

6.2 Z Func

```

vector<int> z_function(string s) {
    int n = (int) s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        // если мы уже видели этот символ
        if (i <= r)
            // то мы можем попробовать его инициализировать z[i - 1]
            // не дальше правой границы: там мы уже ничего не знаем
            z[i] = min(r - i + 1, z[i - 1]);
        // дальше каждое успешное увеличение z[i] сдвинет z-блок на единицу
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        // проверим, правее ли мы текущего z-блока
        if (i + z[i] - 1 > r) {
            r = i + z[i] - 1;
            l = i;
        }
    }
    return z;
}

```

6.3 Manaker

```

//nechet
vector<int> manacher_odd(string s) {
    int n = (int) s.size();
    vector<int> d(n, 1);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r)
            d[i] = min(r - i + 1, d[l + r - i]);
        while (i - d[i] >= 0 && i + d[i] < n && s[i - d[i]] == s[i + d[i]])
            d[i]++;
        if (i + d[i] - 1 > r)
            l = i - d[i] + 1, r = i + d[i] - 1;
    }
    return d;
}

//chet
vector<int> manacher_even(string s) {
    int n = (int) s.size();
    vector<int> d(n, 0);
    int l = -1, r = -1;
    for (int i = 0; i < n - 1; i++) {
        if (i < r)
            d[i] = min(r - i, d[l + r - i - 1]);
        while (i - d[i] >= 0 && i + d[i] + 1 < n && s[i - d[i]] == s[i + d[i] + 1])
            d[i]++;
        if (i + d[i] > r)
            l = i - d[i] + 1, r = i + d[i];
    }
    return d;
}

```

6.4 Aho Korasik

```

const int k = 26;

struct Vertex {
    Vertex *to[k] = {0}, *go[k] = {0};
    Vertex *link = 0, *p;

```

```

    int pch;
    Vertex (int _pch, Vertex *_p) { pch = _pch, p = _p; }
};

Vertex *root = new Vertex(-1, 0);
void add_string(string &s) {
    Vertex *v = root;
    for (char _c : s) {
        int c = int(_c - 'a');
        if (!v->to[c])
            v->to[c] = new Vertex(c, v);
        v = v->to[c];
    }
}
// нам нужно объявить две функции, ссылающиеся друг на друга
// в C/C++
// для этого нужноsignature хотя бы одной из них объявить перед другой
Vertex* go(Vertex *v, int c);

Vertex* link(Vertex *v) {
    if (!v->link) {
        // для строк длиной меньше двух суффиксная ссылка это корень
        if (v == root || v->p == root)
            v->link = root;
        else // в остальных случаях формула работает
            v->link = go(link(v->p), v->pch);
    }
    return v->link;
}

Vertex* go(Vertex *v, int c) {
    if (!v->go[c]) {
        // если переход есть, то автоматный должен вести туда же
        if (v->to[c])
            v->go[c] = v->to[c];
        // если переходанет, но вершина корень,
        // то нужно сделать петлю
        else if (v == root)
            v->go[c] = root;
        else // в остальных случаях формула работает
            v->go[c] = go(link(v), c);
    }
    return v->go[c];
}

```

6.5 Palindrom Tree

```

const int maxn = 1e5, k = 26;

int s[maxn], len[maxn], link[maxn], to[maxn][k];

int n, last, sz;

void init() {
    s[n++] = -1;
    link[0] = 1;
    len[1] = -1;
    sz = 2;
}

int get_link(int v) {
    while (s[n-len[v]-2] != s[n-1])
        v = link[v];
    return v;
}

void add_char(int c) {
    s[n++] = c;
    last = get_link(last);
    if (!to[last][c]) {
        len[sz] = len[last] + 2;
        link[sz] = to[get_link(link[last])][c];
        to[last][c] = sz++;
    }
    last = to[last][c];
}

```

6.6 Suf Array

```

// строка -- это последовательность чисел от 1 до размера алфавита
vector<int> suffix_array(vector<int> &s) {
    s.push_back(0); // добавляем нулевой символ в конец строки

```

```

    int n = (int) s.size(),
        cnt = 0, // вспомогательная переменная:
                // счётчик для сортировки
        cls = 0; // количество классов эквивалентности
    vector<int> c(n), p(n);

    map<int, vector<int>> t;
    for (int i = 0; i < n; i++)
        t[s[i]].push_back(i);

    // нулевой этап
    for (auto &x : t) {
        for (int u : x.second)
            c[u] = cls, p[cnt++] = u;
        cls++;
    }

    // пока все суффиксы не стали уникальными
    for (int l = 1; cls < n; l++) {
        vector<vector<int>> a(cls); // массив для сортировки под счёт
        vector<int> _c(n); // новые классы эквивалентности
        int d = (1 << l) / 2;
        int _cls = cnt = 0; // новое количество классов

        for (int i = 0; i < n; i++) {
            int k = (p[i] - d + n) % n;
            a[c[k]].push_back(k);
        }

        for (int i = 0; i < cls; i++) {
            for (size_t j = 0; j < a[i].size(); j++) {
                // если суффикс начинается с нового класса эквивалентности
                if (j == 0 || c[a[i][j] + d] % n != c[a[i][j - 1] + d] % n)
                    _cls++;
                _c[a[i][j]] = _cls - 1;
                p[cnt++] = a[i][j];
            }
        }

        c = _c;
        cls = _cls;
    }

    return vector<int>(p.begin() + 1, p.end());
}

vector<int> calc_lcp(vector<int> &val, vector<int> &c,
    vector<int> &p) {
    int n = val.size();
    int l = 0; // lcp текущего элемента
    vector<int> lcp(n);
    for (int i = 0; i < n; i++) {
        if (c[i] == n - 1)
            continue;
        int nxt = p[c[i] + 1];
        while (max(i, nxt) + 1 < n && val[i + 1] == val[
            nxt + 1])
            l++;
        lcp[c[i]] = l;
        l = max(0, l - 1);
    }
    return lcp;
}

```

7 Structs

7.1 Fenwick tree

```

vector<int> t;
int n;

void init (int nn)
{
    n = nn;
    t.assign (n, 0);
}

int sum (int r)
{
    int result = 0;

```

```

    for (; r >= 0; r = (r & (r+1)) - 1)
        result += t[r];
    return result;
}

void inc (int i, int delta)
{
    for (; i < n; i = (i | (i+1)))
        t[i] += delta;
}

int sum (int l, int r)
{
    return sum (r) - sum (l-1);
}

void init (vector<int> a)
{
    init ((int) a.size());
    for (unsigned i = 0; i < a.size(); i++)
        inc (i, a[i]);
}

```

7.2 Sparse_table

```

struct sparse{
    vector<vector<int>> dp;
    vector<int> fstbit;
    vector<int> a;
    sparse(vector<int>& v){
        a = v;
        ll n = a.size();
        ll sz = 1;
        ll hgt = 0;
        while(sz < n){
            sz *= 2;
            hgt++;
        }
        dp.assign(hgt, vector<int>(sz, INF));
        fstbit.assign(2*sz, 0);
        fstbit[1] = 0;
        for(int i = 2; i < sz; i++){
            fstbit[i] = fstbit[i >> 1] + 1;
        }
        a.resize(sz, INF);

        for(int h = 0; h < hgt; h++){
            int len = (1 << h);
            for(int mid = len; mid < sz; mid += 2*len){
                dp[h][mid] = mid;
                for(int i = mid+1; i < mid+len; i++){
                    if(a[dp[h][i-1]] > a[i])
                        dp[h][i] = i;
                    else
                        dp[h][i] = dp[h][i-1];
                }
                dp[h][mid-1] = mid-1;
                for(int i = mid-2; i > mid-len-1; i--){
                    if(a[dp[h][i+1]] > a[i])
                        dp[h][i] = i;
                    else
                        dp[h][i] = dp[h][i+1];
                }
            }
        }
        ll get_min(ll l, ll r){
            // if(l == r) return dp[0][l];
            ll h = 63 - __builtin_clzll((1^r));
            if(a[dp[h][l]] > a[dp[h][r]])
                return dp[h][r];
            return dp[h][l];
        }
    };
};

```

7.3 Kurevo

```
mt19937_64 rnd(time(0));
```

```

struct TTreap {
    struct TNode {
        int y;
        int sz;
        int value;
    };
};

```

```

TNode* l = nullptr;
TNode* r = nullptr;

TNode (int value) : value(value), y(rnd()), sz(1) {}

int getSz(TNode* v) {
    return v ? v->sz : 0;
}

void upd(TNode* v) {
    v->sz = getSz(v->l) + getSz(v->r) + 1;
}

TNode* merge(TNode* l, TNode* r) {
    if (!l) {
        return r;
    }
    if (!r) {
        return l;
    }
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        upd(l);
        return l;
    } else {
        r->l = merge(l, r->l);
        upd(r);
        return r;
    }
}

pair<TNode*, TNode*> split(TNode* v, int k) {
    if (!v) {
        return {0, 0};
    }
    if (getSz(v->l) >= k) {
        auto [ll, lr] = split(v->l, k);
        v->l = lr;
        upd(v);
        return {ll, v};
    } else {
        auto [rl, rr] = split(v->r, k - getSz(v->l) - 1);
        v->r = rl;
        upd(v);
        return {v, rr};
    }
}

TNode* root = nullptr;

void pushBack(int x) {
    if (!root) {
        root = new TNode(x);
    } else {
        root = merge(root, new TNode(x));
    }
}

TTreap(const vector<int>& a) {
    for (int i = 0; i < a.size(); i++) {
        pushBack(a[i]);
    }
}

void doFunc(int ql, int qr) {
    auto [l, mr] = split(root, ql);
    // mr = [ql..qr][suffix] (no prefix!!!)
    auto [m, r] = split(mr, qr - ql + 1);

    root = merge(m, l);
    root = merge(root, r);
}

void dfs(TNode* v) {
    if (!v) {
        return;
    }
    dfs(v->l);
    cout << v->value << ' ';
    dfs(v->r);
}

```

```
void print() {  
    dfs(root);  
    cout << '\n';  
};  
};
```